

Claude Code 완전 가이드

커맨드(Commands) · 스킬(Skills) · 서브 에이전트(Subagents) — 개념, 생성 방법, 사용 방법

목차

- 1. 전체 개요와 세 가지 개념의 관계
- 2. 커맨드 (슬래시 커맨드)
 - a. 정의와 종류
 - b. 워크플로우 단계별 주요 커맨드
 - c. 전체 커맨드 표
- 3. 스킬 (Skills)
 - a. 정의와 특징
 - b. 스킬 생성 방법 (단계별)
 - c. 저장 위치와 우선순위
 - d. SKILL.md Frontmatter 필드 전체
 - e. 인자(Arguments) 전달
 - f. 동적 컨텍스트 주입
 - g. 서브 에이전트에서 스킬 실행
 - h. 스킬 호출 권한 제어
 - i. 스킬 테스트와 평가
 - j. 공유 및 배포
 - k. 문제 해결
- 4. 서브 에이전트 (Subagents)
 - a. 정의와 목적
 - b. 내장 서브 에이전트
 - c. 서브 에이전트 생성 방법 (단계별)
 - d. 저장 위치와 우선순위
 - e. Frontmatter 필드 전체
 - f. 도구(Tools) 제어
 - g. 권한 모드
 - h. 영구 메모리
 - i. 서브 에이전트 사용 방법 (호출 방식)
 - j. 백그라운드/포그라운드 실행
 - k. Fork(포크) 기능
 - l. 예제 모음
- 5. 세 가지 기능 비교와 선택 가이드
- 6. 참고 자료

1. 전체 개요

Claude Code를 확장하고 커스터마이징하는 핵심 수단은 **커맨드(Commands)**, **스킬(Skills)**, **서브 에이전트(Subagents)** 세 가지입니다. 이 세 개념은 서로 독립적이면서도 긴밀하게 맞물려 있습니다.

- **커맨드**는 세션을 "제어"하는 명령어입니다. `/` 를 입력해 호출하며, 모델 전환, 컨텍스트 정리, 권한 관리처럼 고정된 동작을 실행하거나(내장 커맨드), 프롬프트 기반 스킬 또는 백그라운드로 실행되는 워크플로우를 실행합니다.
- **스킬**은 재사용 가능한 지침(프롬프트) 덩치입니다. 반복적으로 붙여넣던 지시사항, 체크리스트, 절차를 `SKILL.md` 파일로 만들어두면 Claude가 필요할 때 자동으로 불러오거나, 사용자가 `/스킬이름` 으로 직접 호출할 수 있습니다.
- **서브 에이전트**는 별도의 컨텍스트 창(context window)에서 독립적으로 작업하는 전문화된 보조 에이전트입니다. 대량의 로그, 검색 결과, 파일 내용처럼 메인 대화에 넣고 싶지 않은 작업을 위임하고, 요약된 결과만 돌려받습니다.

핵심 관계: 커스텀 커맨드는 스킬 체계로 통합되었습니다. `.claude/commands/deploy.md` 파일과 `.claude/skills/deploy/SKILL.md` 는 동일하게 `/deploy` 커맨드를 만들며 똑같이 동작합니다. 스킬 쪽이 지원 파일 디렉터리, 호출 권한 제어, 서브 에이전트 실행 등 더 많은 기능을 제공하기 때문에 새로 만들 때는 스킬 형식을 권장합니다. 또한 스킬의 frontmatter에 `context: fork` 를 설정하면 그 스킬을 서브 에이전트 안에서 실행할 수 있고, 반대로 서브 에이전트의 frontmatter에 `skills` 필드를 설정하면 특

2. 커맨드 (Commands)

2.1 정의와 종류

커맨드는 Claude Code 세션 내부에서 세션 자체를 제어하는 수단입니다. 모델 전환, 권한 관리, 컨텍스트 정리, 워크플로우 실행 등에 사용됩니다. `/` 만 입력하면 사용 가능한 모든 커맨드 목록이 나타나고, 이어서 글자를 입력하면 필터링됩니다. 커맨드는 메시지의 맨 앞에 있을 때만 인식되며, 커맨드 이름 뒤에 오는 텍스트는 인자(argument)로 전달됩니다.

커맨드는 동작 방식에 따라 세 가지로 나뉩니다.

- **내장 커맨드(Built-in):** 동작이 CLI 코드에 고정되어 있는 커맨드. 예: `/model` , `/compact` , `/permissions`
- **스킬(Skill) 기반 커맨드:** Claude에게 전달되는 프롬프트로 구성되어 Claude가 도구를 사용해 직접 작업을 수행합니다. 예: `/code-review` , `/debug` , `/batch` , `/loop`
- **워크플로우(Workflow) 기반 커맨드:** 여러 서브 에이전트로 작업을 분산시켜 백그라운드에서 실행하는 동적 워크플로우. 예: `/deep-research`

2.2 워크플로우 단계별 주요 커맨드

세션의 흐름에 따라 자주 쓰이는 커맨드는 다음과 같습니다.

- **저장소에서의 첫 세션:** `/init` 으로 초기 `CLAUDE.md` 를 생성하고 `/memory` 로 다듬습니다. `/mcp` 와 `/agents` 로 서버·서브 에이전트를 설정하고, `/permissions` 로 승인 규칙을 정합니다.
- **작업 진행 중:** 큰 변경 전에는 `/plan` 으로 계획 모드에 진입합니다. `/model` 과 `/effort` 로 추론 강도를 조절하고, 대화가 길어지면 `/context` 로 컨텍스트 사용량을 확인한 뒤 `/compact` 로 요약합니다. 사소한 질문은 `/btw` 로 대화 기록에 남기지 않고 물어볼 수 있습니다.
- **병렬 작업:** `/agents` 로 서브 에이전트 구성을 관리하고, `/tasks` 로 현재 세션의 백그라운드 작업을 확인합니다. `/background` 는 세션 전체를 백그라운드로 돌리고, `/batch` 는 전체 코드베이스에 걸친 대규모 변경을 여러 개의 작업 단위로 나누어 각각 별도의 워크트리(worktree)에서 실행합니다.
- **배포 전:** `/diff` 로 변경 사항을 확인하고, `/code-review` 로 버그와 정리 대상을 점검(`--fix` 로 바로 적용 가능)하며, `/review` 는 GitHub PR에 대해 같은 검토를 수행합니다. `/security-review` 는 더 깊은 보안 검토를 수행합니다.
- **세션 사이:** `/clear` 로 새 작업을 시작하되 프로젝트 메모리는 유지하고, `/resume` 과 `/branch` 로 이전 대화로 돌아가거나 분기합니다.
- **문제 발생 시:** `/rewind` 로 코드와 대화를 이전 시점으로 되돌리고, `/doctor` 와 `/debug` 로 설치·런타임 문제를 진단하며, `/feedback` 으로 세션 컨텍스트를 첨부해 버그를 신고합니다.

2.3 자주 쓰이는 커맨드 표

커맨드	설명
<code>/agents</code>	서브 에이전트 구성 관리 (v2.1.198부터 대화형 마법사는 제거되었고, 파일을 직접 요청하거나 편집하라는 안내가 표시됨)
<code>/plan</code> [설명]	프롬프트에서 바로 계획 모드로 진입. 설명을 붙이면 그 작업으로 바로 시작
<code>/model</code> [모델]	AI 모델 전환 및 새 세션 기본값으로 저장. 인자 없으면 선택기 오픈
<code>/effort</code> [레벨 auto]	추론 강도 설정: low, medium, high, xhigh, max, ultracode 등
<code>/compact</code> [지침]	대화 기록을 요약해 컨텍스트 확보. 요약 시 초점을 지정 가능
<code>/context</code> [all]	현재 컨텍스트 사용량을 색상 그리드로 시각화
<code>/clear</code> [이름]	빈 컨텍스트로 새 대화 시작(별칭: <code>/reset</code> , <code>/new</code>)
<code>/mcp</code>	MCP 서버 연결 및 OAuth 인증 관리
<code>/memory</code>	<code>CLAUDE.md</code> 메모리 파일 편집, 자동 메모리(auto memory) 관리
<code>/permissions</code>	허용/확인/거부 규칙 관리 (별칭: <code>/allowed-tools</code>)
<code>/skills</code>	사용 가능한 스킬 목록 표시. <code>t</code> 로 토큰 수 정렬, <code>Space</code> 로 숨김 처리
<code>/code-review</code> [레벨] [--fix]	스킬. 현재 diff를 검토해 버그·정리 대상 확인. <code>--fix</code> 로 자동 적용, <code>ultra</code> 로 클라우드 심층 검토
<code>/batch</code> <지침>	스킬. 대규모 변경을 5~30개 독립 작업 단위로 분해해 병렬 처리(git 저장소 필요)
<code>/debug</code> [설명]	스킬. 디버그 로깅을 활성화하고 세션 로그를 분석
<code>/deep-research</code> <질문>	워크플로우. 웹 검색을 병렬로 진행해 출처가 명시된 보고서 작성
<code>/loop</code> [간격] [프롬프트]	스킬. 세션을 유지한 채 프롬프트를 반복 실행
<code>/fork</code> <지침>	현재 대화 전체를 물려받는 백그라운드 서브 에이전트(포크) 생성
<code>/rewind</code>	코드와 대화를 체크포인트로 되돌리기(별칭: <code>/checkpoint</code> , <code>/undo</code>)
<code>/doctor</code>	설치와 설정을 진단·검증
<code>/reload-skills</code>	세션 도중 추가·변경된 스킬을 재스캔해 재시작 없이 반영

※ 전체 커맨드 목록은 Claude Code 세션 안에서 `/` 만 입력하거나 공식 문서의 Commands 레퍼런스에서 확인할 수 있습니다. 표에 없는 커맨드도 다수 존재합니다 (`/statusline` , `/hooks` , `/sandbox` , `/worktree` 관련, `/desktop` 등).

2.4 나만의 커맨드 만들기

새로운 커맨드를 추가하려면 **스킬(Skill)**을 만들면 됩니다. 방법은 3장의 "스킬 생성 방법"과 동일합니다. 요약하면:

1. `.claude/skills/<커맨드이름>/SKILL.md` (프로젝트 전용) 또는 `~/.claude/skills/<커맨드이름>/SKILL.md` (개인용, 모든 프로젝트) 경로에 디렉터리를 만듭니다.
2. `SKILL.md` 에 YAML frontmatter(설명 등)와 지침 본문을 작성합니다.
3. 디렉터리 이름이 곧 커맨드 이름이 됩니다. 예: `.claude/skills/deploy/SKILL.md` → `/deploy`

기존에 사용하던 `.claude/commands/이름.md` 형식의 파일도 계속 동작합니다. 다만 스킬 디렉터리 형식이 지원 파일 추가, 호출 권한 제어(`disable-model-invocation`) 등 더 많은 기능을 제공하므로, 새로 만들 때는 스킬 형식을 권장합니다. 같은 이름이 겹치면 스킬이 커맨드 파일보다 우선합니다.

3. 스킬 (Skills)

3.1 정의와 특징

스킬은 `SKILL.md` 파일 하나로 정의되는, Claude에게 지시사항을 제공하는 확장 기능입니다. 자주 반복해서 붙여넣는 지침·체크리스트·다단계 절차가 있다면, 혹은 `CLAUDE.md`의 한 섹션이 규칙이 아니라 "절차"로 커져버렸다면 스킬로 만들 시점입니다. `CLAUDE.md`와 달리 스킬 본문은 **호출될 때만** 컨텍스트에 로드되므로, 아무리 긴 참고 자료라도 실제로 쓰일 때까지는 거의 비용이 들지 않습니다.

Claude Code 스킬은 여러 AI 도구에서 공통으로 쓰이는 **Agent Skills** 오픈 표준을 따르며, Claude Code는 여기에 호출 권한 제어, 서브 에이전트 실행, 동적 컨텍스트 주입 등의 기능을 추가로 제공합니다.

3.2 스킬 생성 방법 (단계별 예제)

예시: 커밋되지 않은 git 변경 사항을 요약하고 위험 요소를 짚어주는 `/summarize-changes` 스킬을 만들어 봅시다.

1단계 — 스킬 디렉터리 생성

```
mkdir -p ~/.claude/skills/summarize-changes
```

개인 스킬(`~/.claude/skills/`)은 내 컴퓨터의 모든 프로젝트에서 사용 가능합니다.

2단계 — SKILL.md 작성

모든 스킬은 두 부분으로 구성됩니다: `---` 사이의 YAML frontmatter(언제 사용할지 알려주는 메타데이터)와, 이어지는 마크다운 본문(실제 지침).

```
---
description: 커밋되지 않은 변경 사항을 요약하고 위험 요소를 짚어줍니다.
  사용자가 무엇이 바뀌었는지 묻거나, 커밋 메시지를 원하거나,
  diff 검토를 요청할 때 사용하세요.
---

## 현재 변경 사항

!`git diff HEAD`

## 지침

위 변경 사항을 두세 개의 불릿 포인트로 요약한 뒤,
에러 처리 누락, 하드코딩된 값, 갱신이 필요한 테스트 등
눈에 띄는 위험 요소를 나열하세요. diff가 비어 있다면
커밋되지 않은 변경 사항이 없다고 답하세요.
```

`!`git diff HEAD`` 줄은 **동적 컨텍스트 주입** 문법입니다. Claude Code가 이 명령을 실행하고, Claude가 스킬 내용을 보기 전에 결과 출력으로 그 줄을 대체합니다. 즉 Claude는 항상 최신 diff가 이미 삽입된 지침을 받게 됩니다.

3단계 — 스킬 테스트

git 프로젝트에서 파일을 조금 수정한 뒤 `claude`를 실행합니다. 두 가지 방식으로 테스트할 수 있습니다.

- **자동 호출:** 설명과 일치하는 질문을 합니다. 예: `"내가 뭘 바꿨지?"`
- **직접 호출:** `/summarize-changes`

3.3 저장 위치와 우선순위

위치	경로	적용 범위
엔터프라이즈	관리형 설정(managed settings)	조직 내 모든 사용자
개인(Personal)	<code>~/.claude/skills/<이름>/SKILL.md</code>	내 모든 프로젝트
프로젝트(Project)	<code>.claude/skills/<이름>/SKILL.md</code>	해당 프로젝트만
플러그인(Plugin)	<code><플러그인>/skills/<이름>/SKILL.md</code>	플러그인이 활성화된 곳

같은 이름의 스킬이 여러 레벨에 있으면 **엔터프라이즈 > 개인 > 프로젝트** 순으로 우선합니다. 어느 레벨의 스킬이든 같은 이름의 번들 스킬(bundled skill)보다 우선합니다. 예를 들어 프로젝트의 `code-review` 스킬은 내장된 `/code-review`를 대체합니다. 플러그인 스킬은 `플러그인이름:스킬이름` 형태의 네임스페이스를 쓰므로 다른 레벨과 충돌하지 않습니다.

프로젝트 하위 디렉터리(모노레포의 각 패키지 등)의 `.claude/skills/`도 자동으로 탐색됩니다. 이름이 겹치면 하위 디렉터리 것은 `apps/web:deploy`처럼 경로가 붙은 이름으로 구분되어 둘 다 사용 가능합니다.

스킬 폴더 구조

```
my-skill/
├── SKILL.md           # 메인 지침 (필수)
├── template.md        # Claude가 채워 넣을 템플릿
├── examples/
│   └── sample.md      # 기대 출력 형식을 보여주는 예시
└── scripts/
    └── validate.sh    # Claude가 실행할 수 있는 스크립트
```

SKILL.md 만 필수이며, 나머지 파일은 필요할 때만 로드되므로 상세한 참고 문서나 스크립트를 자유롭게 추가할 수 있습니다. **SKILL.md** 는 500줄 이하로 유지하고, 상세 내용은 별도 파일로 분리하는 것이 좋습니다.

3.4 SKILL.md Frontmatter 필드 전체 다 외우기

필드	필수	설명
name	아니오	스킬 목록에 표시되는 이름. 기본값은 디렉터리 이름
description	권장	스킬의 목적과 사용 시점. Claude가 자동 호출 여부를 판단하는 근거. 핵심 사용 사례를 앞쪽에 배치할 것 (1,536자에서 잘림)
when_to_use	아니오	트리거 문구·예시 요청 등 추가 컨텍스트. description에 이어붙여 같은 글자수 제한 적용
argument-hint	아니오	자동완성에 표시되는 인자 힌트. 예: <code>[issue-number]</code>
arguments	아니오	<code>\$이름</code> 치환을 위한 이름 있는 위치 인자 목록
disable-model-invocation	아니오	<code>true</code> 면 Claude의 자동 호출 차단, 사용자만 <code>/이름</code> 으로 수동 호출 가능. 기본값 false
user-invocable	아니오	<code>false</code> 면 <code>/</code> 메뉴에서 숨김(백그라운드 지식용). 기본값 true
allowed-tools	아니오	이 스킬이 활성화된 동안 승인 없이 쓸 수 있는 도구 목록
disallowed-tools	아니오	이 스킬이 활성화된 동안 사용 불가능한 도구 목록
model	아니오	스킬 활성화 시 사용할 모델. <code>/model</code> 과 같은 값 또는 <code>inherit</code>
effort	아니오	스킬 활성화 시 추론 강도(low/medium/high/xhigh/max)
context	아니오	<code>fork</code> 로 설정하면 포크된 서브 에이전트 컨텍스트에서 실행
agent	아니오	<code>context: fork</code> 일 때 사용할 서브 에이전트 타입
hooks	아니오	이 스킬 생명주기에 한정된 훅(hooks)
paths	아니오	이 스킬이 자동 활성화되는 파일 경로의 글롭 패턴
shell	아니오	인라인 명령 실행에 사용할 셸: <code>bash</code> (기본) 또는 <code>powershell</code>

사용 가능한 문자열 치환

변수	설명
\$ARGUMENTS	호출 시 전달된 인자 전체 문자열
\$ARGUMENTS[N] / \$N	0부터 시작하는 인덱스로 특정 인자에 접근 (<code>\$0</code> 이 첫 번째)
\$이름	<code>arguments</code> frontmatter에 선언한 이름 있는 인자
\${CLAUDE_SESSION_ID}	현재 세션 ID
\${CLAUDE_EFFORT}	현재 추론 강도(low/medium/high/xhigh/max)
\${CLAUDE_SKILL_DIR}	이 스킬의 SKILL.md 가 위치한 디렉터리
\${CLAUDE_PROJECT_DIR}	프로젝트 루트 디렉터리

3.5 호출 권한 제어

기본적으로 사용자와 Claude 모두 어떤 스킬이든 호출할 수 있습니다. 두 개의 frontmatter 필드로 이를 제한할 수 있습니다.

Frontmatter	사용자 호출	Claude 호출	용도
(기본값)	가능	가능	일반 스킬
disable-model-invocation: true	가능	불가능	부작용이 있는 작업(<code>/deploy</code> , <code>/commit</code> 등). Claude가 임의로 배포를 결정하지 않도록 함
user-invocable: false	불가능	가능	사용자가 직접 실행할 액션이 아닌 배경 지식(예: 레거시 시스템 설명)

예시: 사용자만 실행 가능한 배포 스킬

```
---
name: deploy
description: 프로덕션에 애플리케이션을 배포합니다
disable-model-invocation: true
---
```

\$ARGUMENTS를 프로덕션에 배포합니다:

1. 테스트 스위트 실행
2. 애플리케이션 빌드
3. 배포 대상으로 푸시
4. 배포 성공 여부 확인

3.6 인자(Arguments) 전달

사용자와 Claude 모두 스킬 호출 시 인자를 전달할 수 있습니다. `/fix-issue 123` 처럼 실행하면 `$ARGUMENTS` 가 "123"으로 치환됩니다. 위치 기반 접근이 필요하다면 `$0` , `$1` 등을 사용합니다.

```
---
name: migrate-component
description: 컴포넌트를 한 프레임워크에서 다른 프레임워크로 마이그레이션
---
```

`$0` 컴포넌트를 `$1`에서 `$2`로 마이그레이션하세요.
기존 동작과 테스트를 모두 보존하세요.

`/migrate-component searchBar React Vue` 를 실행하면 `$0` =SearchBar, `$1` =React, `$2` =Vue로 치환됩니다.

3.7 동적 컨텍스트 주입

`!`명령어`` 문법은 스킬 내용이 Claude에게 전달되기 전에 셸 명령을 실행합니다. 명령 출력이 자리를 대체하므로, Claude는 명령어 자체가 아니라 실제 데이터를 받게 됩니다.

```
---
name: pr-summary
description: 풀 리퀘스트의 변경 사항을 요약합니다
context: fork
agent: Explore
allowed-tools: Bash(gh *)
---
```

PR 컨텍스트

- PR diff: `!`gh pr diff``
- PR 코멘트: `!`gh pr view --comments``
- 변경된 파일: `!`gh pr diff --name-only``

작업

이 풀 리퀘스트를 요약하세요...

여러 줄 명령은 ````\n!`` 로 여는 펜스 코드 블록을 사용합니다. 이 동작은 `disableSkillShellExecution` 설정으로 조직 차원에서 끌 수 있습니다.

3.8 서브 에이전트에서 스킬 실행하기

frontmatter에 `context: fork` 를 추가하면 스킬이 격리된 환경에서 실행됩니다. 스킬 내용이 그 서브 에이전트를 구동하는 프롬프트가 되며, 메인 대화 기록에는 접근할 수 없습니다. `agent` 필드로 어떤 서브 에이전트 타입(`Explore` , `Plan` , `general-purpose` , 또는 커스텀 서브 에이전트)을 쓸지 지정합니다. 생략 시 `general-purpose` 가 사용됩니다.

`context: fork` 는 "이 API 컨벤션을 사용하라"처럼 지침만 있고 실행할 작업(task)이 없는 스킬에는 적합하지 않습니다. 작업 없이 지침만 전달되면 서브 에이전트가 의미 있는 결과 없이 종료됩니다.

3.9 스킬 호출 권한 제어 (전역)

- 모든 스킬 비활성화: `/permissions` 에서 `Skill` 을 거부(deny) 목록에 추가
- 특정 스킬만 허용/거부: `Skill(commit)` (정확히 일치), `Skill(deploy *)` (접두사 일치) 형태의 권한 규칙 사용
- 개별 스킬 숨김: frontmatter에 `disable-model-invocation: true` 추가 시 Claude의 컨텍스트에서 완전히 제거됨

3.10 스킬 테스트와 평가

스킬이 트리거되는 것을 확인하는 것과, 그 출력이 의도와 맞는지 확인하는 것은 별개입니다. 공식 `skill-creator` 플러그인을 설치하면 이 비교 루프를 자동화할 수 있습니다.

```
/plugin install skill-creator@claude-plugins-official
```

설치 후 `/reload-plugins` 를 실행하고, "skill-creator로 내 summarize-changes 스킬을 평가해줘"처럼 요청하면 테스트 케이스 작성부터 채점, 스킬 유무에 따른 벤치마크 비교, 설명(description) 문구 튜닝까지 안내받을 수 있습니다.

3.11 문제 해결

증상	해결 방법
스킬이 자동으로 트리거되지 않음	description에 사용자가 실제로 쓸 법한 키워드가 포함되어 있는지 확인. "사용 가능한 스킬이 뭐야?"로 목록 확인. <code>--debug</code> 로 YAML 파싱 오류 확인
스킬이 너무 자주 트리거됨	description을 더 구체적으로 작성하거나 <code>disable-model-invocation: true</code> 추가
스킬 설명이 잘림	스킬이 많을수록 설명 예산이 줄어듦. <code>/doctor</code> 로 확인, <code>skillListingBudgetFraction</code> 설정으로 예산 확대 가능

4. 서브 에이전트 (Subagents)

4.1 정의와 목적

서브 에이전트는 특정 유형의 작업을 처리하는 전문화된 AI 보조자입니다. 검색 결과, 로그, 파일 내용처럼 다시 참조하지 않을 내용으로 메인 대화가 넘칠 것 같은 부수 작업이 있다면 서브 에이전트를 사용할 시점입니다. 서브 에이전트는 자체 컨텍스트 창에서 작업하고 요약만 반환합니다.

각 서브 에이전트는 독자적인 컨텍스트 창, 커스텀 시스템 프롬프트, 특정 도구 접근 권한, 독립적인 권한 설정을 가집니다. Claude가 서브 에이전트의 description과 일치하는 작업을 만나면 그 서브 에이전트에 위임하고, 서브 에이전트는 독립적으로 작업한 뒤 결과를 반환합니다.

서브 에이전트는 다음을 도와줍니다:

- **컨텍스트 보존:** 탐색과 구현 작업을 메인 대화 밖에 둬
- **제약 강제:** 서브 에이전트가 사용할 수 있는 도구를 제한
- **설정 재사용:** 사용자 레벨 서브 에이전트로 여러 프로젝트에서 재사용
- **행동 전문화:** 특정 도메인에 집중된 시스템 프롬프트
- **비용 제어:** Haiku 같은 더 빠르고 저렴한 모델로 작업 라우팅

4.2 내장 서브 에이전트

이름	모델	설명
Explore	메인 대화 상속 (API 사용 시 Opus 상한)	읽기 전용, 코드 검색·분석에 최적화. Write/Edit 거부. CLAUDE.md와 git 상태를 생략해 빠르고 저렴함
Plan	메인 대화 상속	계획 모드(Plan Mode)에서 계획을 제시하기 전 컨텍스트를 수집하는 리서치 에이전트. 읽기 전용
general-purpose	메인 대화 상속	탐색과 수정이 모두 필요한 복잡하고 다단계인 작업을 위한 에이전트. 모든 도구 사용 가능
statusline-setup	Sonnet	<code>/statusline</code> 실행 시 상태 표시줄 구성에 사용
claude-code-guide	Haiku	Claude Code 기능에 대한 질문에 답할 때 사용

Explore와 Plan은 CLAUDE.md와 git 상태를 건너뛰어 빠르고 저렴하게 유지되는 유일한 서브 에이전트입니다. 그 외 모든 내장/커스텀 서브 에이전트는 둘 다 로드합니다. 특정 내장 타입을 막으려면 `permissions.deny` 에 `Agent(Explore)` 처럼 추가하면 됩니다.

4.3 서브 에이전트 생성 방법 (단계별)

서브 에이전트는 YAML frontmatter가 있는 마크다운 파일입니다. Claude에게 작성을 요청하거나 직접 파일을 작성할 수 있습니다.

1단계 — Claude에게 생성 요청

```
~/ .claude/agents/ 에 개인용 code-improver 서브 에이전트를 만들어줘.
파일을 스캔해서 가독성, 성능, 베스트 프랙티스 측면의 개선점을
제안하도록 해줘. 각 이슈를 설명하고, 현재 코드를 보여준 뒤,
개선된 버전을 제공하게 해줘. 읽기 전용으로 만들고 Sonnet을 쓰게 해줘.
```

2단계 — 생성된 파일 확인

`~/ .claude/agents/code-improver.md` 를 열어 frontmatter가 요청한 대로인지 확인합니다. 결과는 다음과 같습니다.

```
---
name: code-improver
description: 파일을 스캔해 가독성, 성능, 베스트 프랙티스 측면의
  개선점을 제안합니다. 코드 작성/수정 후 사용하세요.
tools: Read, Grep, Glob
model: sonnet
---
```

당신은 코드 개선 전문가입니다. 발견한 각 이슈에 대해 문제를 설명하고, 현재 코드를 보여준 뒤, 개선된 버전을 제공하세요.

파일이 `~/ .claude/agents/` 에 있으므로 이 서브 에이전트는 내 컴퓨터의 모든 프로젝트에서 사용 가능합니다. 프로젝트 전용으로 제한하려면 해당 프로젝트의 `.claude/agents/` 로 옮기면 됩니다.

3단계 — 사용해보기

```
이 프로젝트에서 code-improver 에이전트를 사용해서 개선점을 제안해줘
```

Claude가 새 서브 에이전트에 위임해 코드베이스를 스캔하고 개선 제안을 반환합니다. 새로 만든 `agents` 디렉터리가 세션 시작 전에 없었다면 Claude Code를 재시작해야 인식됩니다.

4.4 저장 위치와 우선순위

위치	범위	우선순위	생성 방법
관리형 설정(Managed settings)	조직 전체	1 (최고)	관리형 설정으로 배포
<code>--agents</code> CLI 플래그	현재 세션만	2	Claude Code 실행 시 JSON 전달
<code>.claude/agents/</code>	현재 프로젝트	3	Claude에게 요청 또는 직접 파일 생성
<code>~/.claude/agents/</code>	내 모든 프로젝트	4	Claude에게 요청 또는 직접 파일 생성
플러그인의 <code>agents/</code>	플러그인 활성화된 곳	5 (최저)	플러그인 설치

프로젝트 서브 에이전트(`.claude/agents/`)는 특정 코드베이스에 특화된 경우에 적합하며, 버전 관리에 커밋해 팀과 공유할 수 있습니다. **사용자 서브 에이전트**(`~/.claude/agents/`)는 모든 프로젝트에서 쓰는 개인용입니다.

CLI로 즉석 정의하기

```
claude --agents '{
  "code-reviewer": {
    "description": "전문 코드 리뷰어. 코드 변경 후 적극적으로 사용.",
    "prompt": "당신은 시니어 코드 리뷰어입니다. 코드 품질, 보안,
      베스트 프랙티스에 집중하세요.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  }
}'
```

4.5 Frontmatter 필드 전체

필드	필수	설명
<code>name</code>	예	소문자와 하이픈으로 된 고유 식별자
<code>description</code>	예	Claude가 언제 이 서브 에이전트에 위임할지 판단하는 근거
<code>tools</code>	아니오	사용 가능한 도구 목록. 생략 시 모든 도구 상속
<code>disallowedTools</code>	아니오	상속받은 목록에서 제거할 도구
<code>model</code>	아니오	<code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code> , 전체 모델 ID, 또는 <code>inherit</code> . 기본값 <code>inherit</code>
<code>permissionMode</code>	아니오	<code>default</code> , <code>acceptEdits</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , <code>plan</code>
<code>maxTurns</code>	아니오	서브 에이전트가 멈추기 전 최대 에이전트 턴 수
<code>skills</code>	아니오	시작 시 컨텍스트에 미리 로드할 스킬 목록(설명뿐 아니라 전체 내용 주입)
<code>mcpServers</code>	아니오	이 서브 에이전트가 접근 가능한 MCP 서버
<code>hooks</code>	아니오	이 서브 에이전트 생명주기에 한정된 훅
<code>memory</code>	아니오	영구 메모리 범위: <code>user</code> , <code>project</code> , <code>local</code>
<code>background</code>	아니오	<code>true</code> 면 항상 백그라운드 작업으로 실행
<code>effort</code>	아니오	이 서브 에이전트 활성화 시 추론 강도
<code>isolation</code>	아니오	<code>worktree</code> 로 설정 시 임시 git 워크트리에서 격리 실행
<code>color</code>	아니오	작업 목록·트랜스크립트에서의 표시 색상
<code>initialPrompt</code>	아니오	메인 세션 에이전트로 실행될 때 자동 제출되는 첫 사용자 턴

4.6 도구(Tools) 제어

기본적으로 서브 에이전트는 메인 대화에서 사용 가능한 도구를 상속합니다. `tools` 필드는 허용 목록, `disallowedTools` 는 거부 목록으로 작동합니다.

```
---
name: safe-researcher
description: 제한된 기능을 가진 리서치 에이전트
tools: Read, Grep, Glob, Bash
---
```

둘 다 설정되면 `disallowedTools` 가 먼저 적용된 뒤 `tools` 가 남은 풀에 대해 계산됩니다. `mcp__<서버>` 나 `mcp__<서버>__*` 같은 MCP 서버 레벨 패턴도 사용 가능합니다.

4.7 권한 모드

모드	동작
default	표준 권한 확인, 프롬프트 표시
acceptEdits	파일 편집과 일반적인 파일시스템 명령을 자동 승인
auto	자동 모드: 백그라운드 분류기가 명령과 보호된 디렉터리 쓰기를 검토
dontAsk	권한 프롬프트를 자동 거부(명시적으로 허용된 도구는 계속 동작)
bypassPermissions	권한 프롬프트를 건너뛴. 주의해서 사용
plan	계획 모드 (읽기 전용 탐색)

4.8 영구 메모리

memory 필드는 대화 간에 유지되는 영구 디렉터리를 서브 에이전트에 부여합니다. 코드베이스 패턴, 디버깅 인사이트, 아키텍처 결정 등을 시간이 지나며 축적할 수 있습니다.

범위	위치	사용 시점
user	~/.claude/agent-memory/<이름>/	모든 프로젝트에서 학습 내용을 기억해야 할 때
project	.claude/agent-memory/<이름>/	프로젝트별 지식을 버전 관리로 공유할 때 (권장 기본값)
local	.claude/agent-memory-local/<이름>/	프로젝트별이지만 버전 관리에는 포함하지 않을 때

4.9 서브 에이전트 사용 방법 (호출 방식)

자동 위임으로 충분하지 않을 때, 세 가지 방식으로 직접 요청할 수 있으며 강도가 점점 강해집니다.

- **자언어:** 프롬프트에서 서브 에이전트 이름을 언급 — **"test-runner 서브 에이전트를 사용해서 실패하는 테스트를 고쳐줘"**
- **@-멘션:** @ 를 입력하고 자동완성에서 선택 — 특정 서브 에이전트가 확실히 실행되도록 보장
- **세션 전체 적용:** **claude --agent code-reviewer** 로 세션 전체가 그 서브 에이전트의 시스템 프롬프트·도구 제한·모델을 사용

자동 위임을 유도하려면 서브 에이전트의 description에 "적극적으로 사용(use proactively)" 같은 문구를 포함시키세요.

4.10 백그라운드/포그라운드 실행

포그라운드 서브 에이전트는 완료될 때까지 메인 대화를 블록하며 권한 프롬프트가 그대로 전달됩니다. **백그라운드** 서브 에이전트는 계속 작업하는 동안 동시에 실행되며, 권한이 필요한 도구 호출에 도달하면 프롬프트가 메인 세션에 표시됩니다(어떤 서브 에이전트인지 이름과 함께). 최근 버전부터는 서브 에이전트가 기본적으로 백그라운드에서 실행됩니다. **Ctrl+B** 로 실행 중인 작업을 백그라운드로 보낼 수 있습니다.

4.11 Fork(포크) — 대화 전체를 물려받는 서브 에이전트

포크는 처음부터 시작하는 대신 **전체 대화 기록을 물려받는** 서브 에이전트입니다. 일반 서브 에이전트가 주는 입력 격리가 없어지는 대신, 메인 세션과 같은 시스템 프롬프트·도구·모델·메시지 기록을 그대로 보므로 상황을 다시 설명할 필요가 없습니다. 포크 자신의 도구 호출은 메인 대화에 남지 않고 최종 결과만 돌아옵니다.

```
/fork 지금까지의 파서 변경 사항에 대한 단위 테스트 초안을 작성해줘
```

포크는 프롬프트 입력창 아래 패널에 나타나 백그라운드로 실행되며, 완료되면 결과가 메인 대화에 메시지로 도착합니다.

4.12 예제 모음

코드 리뷰어 (읽기 전용)

```
---
name: code-reviewer
description: 전문 코드 리뷰 담당. 코드 품질, 보안, 유지보수성을
  적극적으로 검토합니다. 코드 작성/수정 직후 사용하세요.
tools: Read, Grep, Glob, Bash
model: inherit
---
```

당신은 높은 수준의 코드 품질과 보안을 보장하는 시니어 코드 리뷰어입니다.

호출되면:

1. git diff로 최근 변경 사항 확인
2. 수정된 파일에 집중
3. 즉시 검토 시작

검토 체크리스트:

- 코드가 명확하고 읽기 쉬운가
- 함수와 변수 이름이 적절한가
- 중복 코드가 없는가
- 적절한 에러 처리
- 노출된 비밀 키나 API 키가 없는가
- 입력 검증 구현 여부
- 테스트 커버리지
- 성능 고려 사항

우선순위별로 정리해 피드백을 제공하세요: 치명적 이슈(반드시 수정), 경고(수정 권장), 제안(개선 고려).

디버거 (읽기+수정)

```
---
name: debugger
description: 에러, 테스트 실패, 예상치 못한 동작에 대한 디버깅
  전문가. 어떤 문제든 발생 시 적극적으로 사용하세요.
tools: Read, Edit, Bash, Grep, Glob
---
```

당신은 근본 원인 분석을 전문으로 하는 뛰어난 디버거입니다.

- 호출되면:
1. 에러 메시지와 스택 트레이스 확보
 2. 재현 단계 파악
 3. 실패 지점 격리
 4. 최소한의 수정 구현
 5. 해결책 검증

각 이슈에 대해 다음을 제공하세요: 근본 원인 설명, 진단을 뒷받침하는 증거, 구체적인 코드 수정, 테스트 방법, 재발 방지 권장 사항.

읽기 전용 데이터베이스 검증기 (Hooks 사용)

```
---
name: db-reader
description: 읽기 전용 데이터베이스 쿼리 실행. 데이터 분석이나
  보고서 생성 시 사용하세요.
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

당신은 읽기 전용 접근 권한을 가진 데이터베이스 분석가입니다. 데이터에 대한 질문에 답하기 위해 SELECT 쿼리를 실행하세요.

이 예제는 PreToolUse 혹은 Bash 명령을 실행 전 검증해 INSERT , UPDATE , DELETE 등 쓰기 작업을 차단하고 오직 SELECT 쿼리만 허용합니다. 검증 스크립트가 종료 코드 2를 반환하면 명령이 차단되고 에러 메시지가 Claude에게 전달됩니다.

5. 세 가지 기능 비교와 선택 가이드

항목	커맨드	스킬	서브 에이전트
본질	세션 제어 동작 실행	재사용 가능한 지침(프롬프트)	독립 컨텍스트를 가진 전문 작업자
실행 위치	내장 로직 또는 메인 대화	메인 대화 컨텍스트 (또는 <code>context: fork</code> 로 격리)	별도의 격리된 컨텍스트 창
파일 형식	내장(코드) 또는 스킬	<code>SKILL.md</code> (YAML frontmatter + 마크다운)	<code>.md</code> 파일 (YAML frontmatter + 시스템 프롬프트)
저장 위치	-	<code>.claude/skills/</code> , <code>~/.claude/skills/</code>	<code>.claude/agents/</code> , <code>~/.claude/agents/</code>
호출 방식	<code>/커맨드</code>	자동(설명 매칭) 또는 <code>/스킬이름</code>	자동 위임, 자연어, <code>@멘션</code> , <code>--agent</code>
주된 용도	모델 전환, 컨텍스트 관리, 고정 워크플로우	반복 지침, 체크리스트, 도메인 지식, 간단한 작업 자동화	대량 출력이 나오는 조사·검색, 격리가 필요한 작업, 도구 제한이 필요한 작업

선택 팁: 재사용 가능한 프롬프트나 워크플로우가 필요하며 메인 대화 컨텍스트 안에서 실행되어도 괜찮다면 **스킬**을 선택하세요. 검색 결과나 로그처럼 방대한 출력이 나오는 부수 작업을 메인 컨텍스트에서 격리하고 싶다면, 또는 도구 접근을 엄격히 제한하고 싶다면 **서브 에이전트**를 선택하세요. 둘은 상호 배타적이지 않습니다 — 스킬의 `context: fork` 로 서브 에이전트 안에서 스킬을 실행하거나, 서브 에이전트의 `skills` 필드로 스킬을 미리 로드해 결합할 수 있습니다.

6. 참고 자료

- 커맨드 레퍼런스: <https://code.claude.com/docs/en/commands>
- 스킬 가이드: <https://code.claude.com/docs/en/skills>
- 서브 에이전트 가이드: <https://code.claude.com/docs/en/sub-agents>
- 전체 문서 지도: https://code.claude.com/docs/en/claude_code_docs_map.md
- 플러그인: <https://code.claude.com/docs/en/plugins>
- 훅(Hooks): <https://code.claude.com/docs/en/hooks>

본 문서는 2026년 7월 기준 Claude Code 공식 문서(code.claude.com)를 바탕으로 작성되었습니다. Claude Code는 계속 업데이트되므로 최신 세부사항은 공식 문서를 참고하시기 바랍니다.